L05. Logic Minimization

Lecture 05

5.1 2단 논리 단순화 복습과 예제

> **PI = 더 이상 확장할 수 없는 implicant. EPI = on-set minterm 하나를 덮는 PI 가 그것 하나뿐일 때의 그 PI.**

- 목표: 곱항 수 최소. 같으면 리터럴 수 최소.
- 불리언 큐브: 부울식을 시각화, 축 하나가 변수 하나. 카르노 맵: 다차원 공간을 2차원에 투영해 인접한 minterm(maxterm)을 합침.

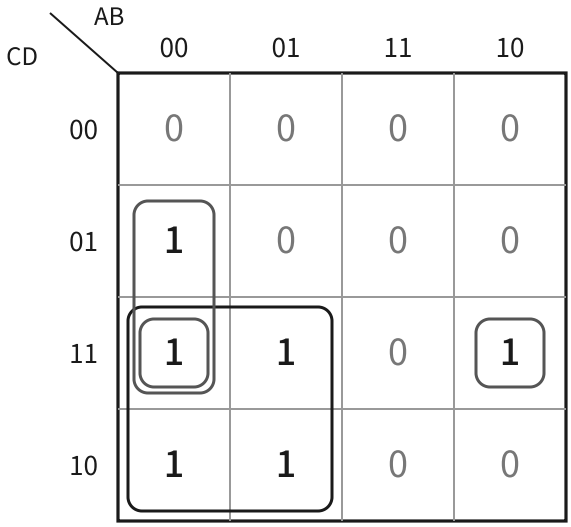
예: 2비트 비교기

2비트 수 AB, CD 비교. 출력은 F_{eq}(AB = CD), F_{lt}(AB < CD), F_{gt}(AB > CD).

A	B	C	D	F_{eq}	F_{lt}	F_{gt}
0	0	0	0	1	0	0
0	0	0	1	0	1	0
0	0	1	0	0	1	0
0	0	1	1	0	1	0
0	1	0	0	0	0	1
0	1	0	1	1	0	0
0	1	1	0	0	1	0
0	1	1	1	0	1	0
A	**B**	**C**	**D**	**F_{eq}**	**F_{lt}**	**F_{gt}**
1	0	0	0	0	0	1
1	0	0	1	0	0	1

A	B	C	D	F_{eq}	F_{lt}	F_{gt}
1	0	1	0	1	0	0
1	0	1	1	0	1	0
1	1	0	0	0	0	1
1	1	0	1	0	0	1
1	1	1	0	0	0	1
1	1	1	1	1	0	0

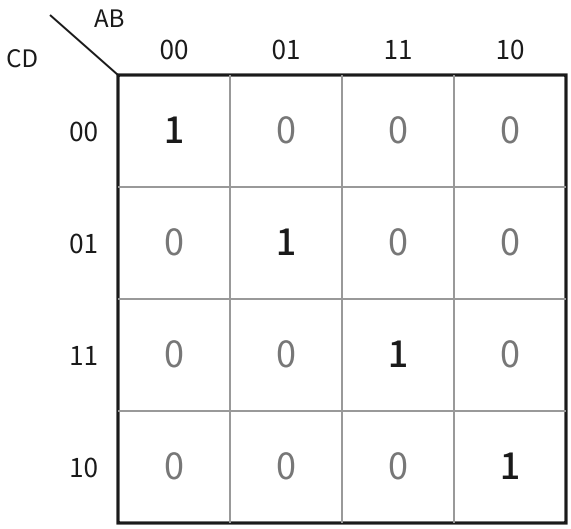
F_{lt}



F_{lt} = Σm(1,2,3,6,7,11). 묶음은 A′C(m2,3,6,7), A′B′D(m1,3), B′CD(m3,11).

F_{lt} = A′C + A′B′D + B′CD

F_{eq}



F_{eq} = Σm(0,5,10,15). 1이 대각선에만 있고 서로 인접하지 않아 묶이지 않는다.

F_{eq} 인수분해 — 먼저 진리표의 on-set에서 SOP를 구한다.

F_{eq} = A′B′C′D′ + A′BC′D + AB′CD′ + ABCD

A와 C의 값이 같은 항끼리 묶으면 F_{eq} = A′C′(B′D′ + BD) + AC(B′D′ + BD)

공통 인수 (B′D′ + BD)를 추출하면 F_{eq} = (A′C′ + AC)(B′D′ + BD)

XNOR의 정의 X XNOR Y = X′Y′ + XY를 적용하면 F_{eq} = (A XNOR C)(B XNOR D)

이 최종식은 각 비트쌍의 일치 여부를 XNOR로 계산한 뒤 두 결과를 AND하는 구조이다. 최소 SOP 표현과 XNOR을 이용한 인수분해 표현을 구별한다.

이 비교기에서 F_{eq}의 1들은 K-map에서 서로 인접하지 않으므로 최소 SOP는 네 개의 4리터럴 minterm으로 구성된다. 그러나 두 2비트 수가 같으려면 A=C와 B=D가 동시에 성립해야 하므로, F_{eq}를 (A XNOR C)(B XNOR D)로 인수분해할 수 있다. 이는 XNOR 게이트를 사용할 때 간결한 표현이며, 최소 SOP와는 구별해야 한다. K-map의 1들이 묶이지 않는 모든 함수가 XNOR 곱으로 인수분해되는 것은 아니다.

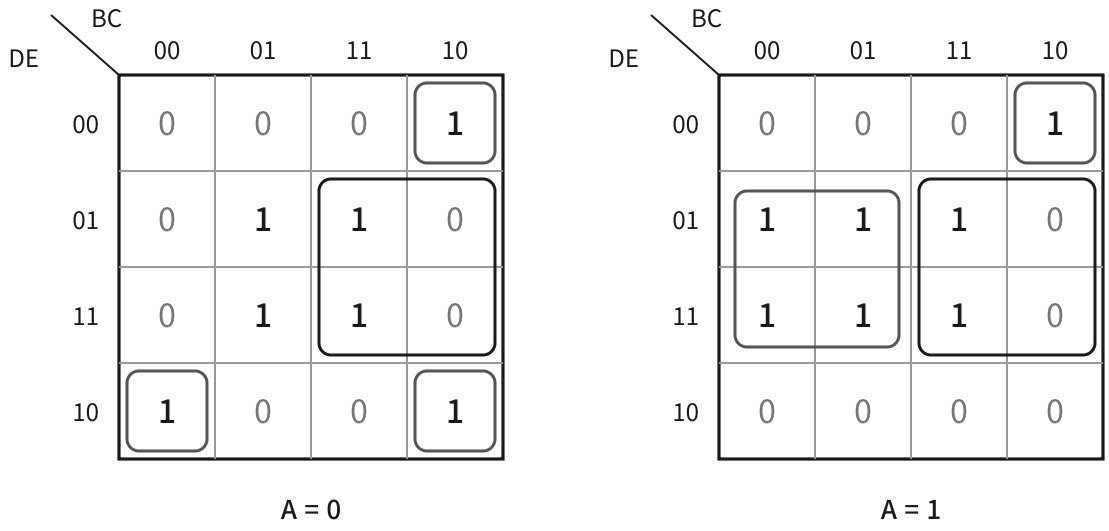
F_{gt}

F_{gt} = AC′ + BC′D′ + ABD′. 특히 F_{gt} = Σm(4,8,9,12,13,14)이고, AB > CD라는 조건을 직접 최소화하거나 F_{lt}의 두 비교 대상을 서로 바꾸어 동일한 결과를 얻을 수 있다.

5변수 K-map

- 변수가 5개면 4변수 맵 두 개(A = 0, A = 1)로 그림. 두 맵의 같은 위치에 있는 셀끼리 인접하므로 묶음이 맵을 건너갈 수 있음.

예 · F(A,B,C,D,E) = Σm(2,5,7,8,10,13,15,17,19,21,23,24,29,31)



열 BC, 행 DE 인 4변수 맵 두 개(A = 0, A = 1). 같은 위치의 셀끼리 인접.

- CE: 두 맵 모두에서 BC ∈ 01, 11 이고 DE ∈ 01, 11 인 8셀 묶음 (m5, 7, 13, 15, 21, 23, 29, 31).
- AB′E: A = 1 맵에서 BC ∈ 00, 01 이고 DE ∈ 01, 11 (m17, 19, 21, 23).
- A′C′DE′: A = 0 맵의 양 끝 열(BC = 00, 10)은 말려서 인접. DE = 10 (m2, 10).
- BC′D′E′: BC = 10, DE = 00 셀을 두 맵에 걸쳐 묶음 (m8, 24).

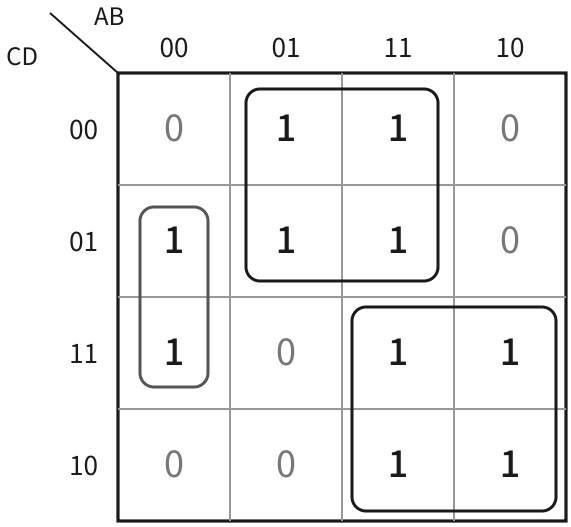
F = CE + AB′E + A′C′DE′ + BC′D′E′

5.2 최소화 과정의 형식화

Implicant	on-set 의 1 하나, 또는 K-map 에서 하나의 곱항으로 합칠 수 있는 1들의 묶음. 0 인 칸은 덮지 않음.
Prime implicant (PI)	다른 implicant 와 합쳐서 리터럴을 더 제거할 수 없는 implicant (maximally reduced).

Essential prime implicant (EPI)	on-set 원소 하나를 덮는 PI 가 단 하나뿐이면, 그 PI 가 EPI.

예 1 · F = Σm(1,3,4,5,10,11,12,13,14,15)



AC 와 BC′ 가 EPI. A′B′D 는 남은 m1, m3 을 덮는다.

PI	덮는 minterm	EPI
AB	12, 13, 14, 15	아니오
AC	10, 11, 14, 15	예
A′B′D	1, 3	아니오
A′C′D	1, 5	아니오
BC′	4, 5, 12, 13	예
B′CD	3, 11	아니오

PI 6개, EPI 2개(AC, BC′). EPI 가 못 덮은 m_1, m_3 은 A′B′D 로 덮음. A′C′D 로는 m_3 이 남음.

(minimized) F = AC + BC′ + A′B′D

예 2 · F = Σm(1,5,6,7,11,12,13,15)

PI	포함 minterm	EPI
ABC′	12, 13	예
ACD	11, 15	예
A′BC	6, 7	예
A′C′D	1, 5	예
BD	5, 7, 13, 15	아니오

PI 5개, EPI 4개. BD 는 PI 지만 EPI 아님. minterm 이 전부 EPI 로 덮여서 BD 불필요.

(minimized) F = A′C′D + ABC′ + ACD + A′BC

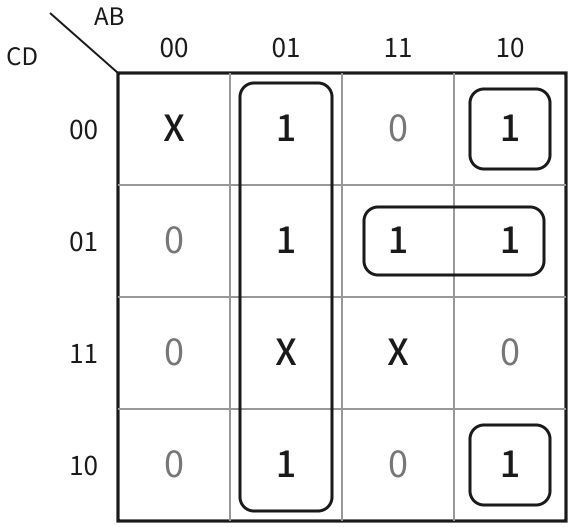
K-map 에서 최소식 구하기

Step 1. 1과 X 의 maximal PI을 전부 찾기.

Step 2. EPI 전부 찾기.

Step 3. EPI 로 안 덮인 1을 최소 개수의 PI 로 덮기.

예 · F = Σm(4,5,6,8,9,10,13) + d(0,7,15)



X 는 PI 를 키우는 데는 쓰지만 덮어야 할 칸은 아니다.

PI	덮는 1	EPI
AB′C′	8, 9	아니오
AB′D′	8, 10	예
AC′D	9, 13	아니오
A′B	4, 5, 6	예
A′C′D′	4	아니오
BD	5, 13	아니오
B′C′D′	8	아니오

EPI 둘(A′B, AB′D′)이 4, 5, 6, 8, 10 을 덮음. 남은 9, 13 은 AC′D 하나로 해결.

F = A′B + AB′D′ + AC′D

K-map 최소화

최소 SOP: F = AB′D′ + A′B + AC′D

1과 X를 묶어 곱항을 만든다. X는 묶음에 도움이 될 때만 쓴다.

5.3 2단 논리 단순화의 자동화

Quine-McCluskey(Q-M) 방법	exact 방법. 최소화에 시간이 아주 오래 걸림.
Espresso	휴리스틱 방법. 빠르고 꽤 좋은 해를 주지만 최적이 아닐 수 있음.

Q-M 방법

- minterm 식을 최소형으로 줄이는 정확한 알고리즘. 컴퓨터로 프로그래밍 가능.
- K-map 은 시각적 패턴에 의존해서 사람한테는 쉽지만 자동화가 어려움. Q-M 은 절차가 정해져 있고 단계마다 표를 쓰므로 프로그램으로 구현하기 훨씬 낫다.
- K-map 보다 많은 변수까지 처리 가능 (슬라이드: ~ 21 variables).
- 2단계 절차. ① uniting theorem XY + XY′ = X 를 반복해서 minterm 식의 PI 를 전부 찾음. ② prime implicant chart(PI chart)로 EPI 를 찾고, 남은 minterm 을 덮는 최소 PI 집합을 고름.

Step 1: 모든 PI 찾기

- 두 곱항이 한 변수만 다르면 합침. ABC + ABC′ = AB.
- 컴퓨터에서는 cube notation. ABC = 111, ABC′ = 110, 합치면 11− (− 는 줄어든 변수). 1의 개수는 3 과 2.
- 한 변수만 다른 두 곱항은 1 의 개수가 정확히 1 차이. 따라서 1 의 개수가 1 차이 나는 곱항끼리만 합칠 수 있음. 곱항을 1 의 개수별로 그룹 지음.

F(A,B,C,D) = Σm(0,1,2,5,6,7,8,9,10,14), 10개의 minterm

① minterm 전개 ② 1의 개수로 그룹 나누기. 그룹 i = 1이 i개인 곱항. 이웃한 그룹(i, i±1)끼리만 합칠 수 있음.

그룹	minterm	cube
0	m0	0000
1	m1	0001
	m2	0010
	m8	1000
2	m5	0101
	m6	0110
	m9	1001
	m10	1010
3	m7	0111
	m14	1110

③ 한 자리만 다른 두 항을 합침. 다른 자리는 −(dash), 합쳐진 minterm 은 체크(✓).

size 2 implicant	cube	size 4 implicant	cube
m(0,1)	000−	m(0,1,8,9)	−00−
m(0,2)	00−0	m(0,2,8,10)	−0−0
m(0,8)	−000	m(2,6,10,14)	−−10
m(1,5)	0−01		
m(1,9)	−001		

size 2 implicant	cube	size 4 implicant	cube
m(2,6)	0−10		
m(2,10)	−010		
m(8,9)	100−		
m(8,10)	10−0		
m(5,7)	01−1		
m(6,7)	011−		
m(6,14)	−110		
m(10,14)	1−10		

- size 2 → 4: dash 를 0, 1 과 다른 세 번째 값으로 취급. dash 위치가 같고 나머지 비트 중 한 비트만 다른 두 항을 합침.
- 합치는 순서만 달라 같은 항이 또 나오면 redundant. 하나만 남김. m(0,1,8,9) 와 m(0,8,1,9), m(2,6,10,14) 와 m(2,10,6,14) 가 그 경우.
- 더 못 합칠 때까지 반복. 체크 안 된 항이 전부 PI. 여기서는 m(1,5) 0−01, m(5,7) 01−1, m(6,7) 011−, m(0,1,8,9) −00−, m(0,2,8,10) −0−0, m(2,6,10,14) −−10.

PI: B′C′, B′D′, CD′, A′C′D, A′BD, A′BC

PI 를 전부 더한 F = B′C′ + B′D′ + CD′ + A′C′D + A′BD + A′BC 는 최소형이 아니라 PI 모음. 최소형은 PI chart 로 찾음.

Step 2: PI chart 로 최소 PI 집합 고르기

PI		m0	m1	m2	m5	m6	m7	m8	m9	m10	m14
m(0,1,8,9)	B′C′	×	×					×	×		
m(0,2,8,10)	B′D′	×		×				×		×	

PI		m0	m1	m2	m5	m6	m7	m8	m9	m10	m14
m(2,6,10,14)	CD′			×		×				×	×
m(1,5)	A′C′D		×		×						
m(5,7)	A′BD				×		×				
m(6,7)	A′BC					×	×				

- 첫 열은 PI, 나머지 열은 PI 가 덮는 on-set 원소.
- **EPI 찾기**: 한 PI 만 덮는 minterm 열(distinguished 1-cell)이 있으면 그 PI 가 EPI. m9 는 B′C′ 만, m14 는 CD′ 만 덮음. EPI = {B′C′, CD′}.
- EPI 가 덮는 minterm 을 지우고 남은 걸 덮는 최소 PI 선택. 남는 건 m5, m7 뿐이고 A′BD 하나로 둘 다 덮임. 최소 PI 집합 = {A′BD}.

Step 3. EPI를 모두 선택하고 이들이 덮는 on-set minterm 열을 PI chart에서 제거한다. 남은 열이 없으면 선택을 종료한다. 남은 열이 있으면 각 minterm을 덮을 수 있는 PI들의 조합 중 모든 남은 minterm을 덮는 cover를 찾는다. 정확한 선택에는 Petrick's method 또는 가능한 PI 부분집합의 완전 탐색을 사용할 수 있다. Petrick's method에서는 PI i를 선택한다는 불리언 변수 p_i를 도입한다. 각 남은 minterm m_j에 대해 그 minterm을 덮는 PI 선택 변수들의 OR를 만들고, 이 OR들을 모두 AND한 식 $P = \prod_j (\Sigma_{i:\ PI_i \text{ covers } m_j}\ p_i)$을 구성한다. 분배법칙과 흡수법칙으로 식을 정리하여 모든 minterm을 덮는 PI 조합을 구한다. 그중 선택된 PI 수가 최소인 조합을 선택하고, 동률이면 PI들의 전체 리터럴 수가 최소인 조합을 선택한다. EPI를 포함한 이 조합이 최소 SOP cover가 된다. PI를 가장 많이 덮는 순서로 탐욕적으로 고르는 방법만으로는 전역 최적성이 보장되지 않는다.

이 예제에 적용하면 EPI인 B′C′와 CD′가 덮는 열을 제거한 뒤 남는 minterm은 m5와 m7뿐이다. m5를 덮는 PI는 A′C′D와 A′BD, m7을 덮는 PI는 A′BD와 A′BC이므로 Petrick 식은 $(p_{A'C'D} + p_{A'BD})(p_{A'BD} + p_{A'BC})$가 되고, 이를 정리하면 A′BD 하나를 선택하는 조합이 최소 cover로 남는다.

- {B′C′, CD′, A′BD} 가 minterm 을 전부 덮는 최소 집합.

최소형

F = B′C′ + CD′ + A′BD

Don’t care 가 있을 때

Step 1 (모든 PI 찾기)	don’t care 를 1처럼 취급하고 평소대로.
Step 2 (최소 PI 선택)	PI chart 에서 don’t care 열은 숨기고 평소대로.

F(A,B,C,D) = Σm(0,1,2,5,9,10,14) + Σd(6,7,8)

PI	덮는 1
A′BC	don’t care만 덮음: 최종 cover 선택에서는 제외
A′BD	5
A′C′D	1, 5
B′C′	0, 1, 9
B′D′	0, 2, 10
CD′	2, 10, 14

EPI: B′C′ (m9 는 이 PI 만 덮음), CD′ (m14). 남는 1은 m5 하나, A′C′D 와 A′BD 둘 다 덮음.

선택 기준: 곱항 수가 같으면 리터럴 수 비교. A′C′D, A′BD 모두 3개이므로 어느 쪽이든 최소.

F = B′C′ + CD′ + A′C′D (또는 A′BD)

Q-M 은 K-map 과 기능적으로 같다

- Step 1 의 PI = K-map 에서 찾는 최대 묶음(B′C′, B′D′, CD′, A′C′D, A′BD, A′BC).
- Step 2 의 EPI = K-map 에서 한 묶음만 덮는 1(m9, m14)을 덮는 PI. 남는 m5 는 덮는 PI 중 하나 (A′BD)로 선택.

Espresso

- Espresso = 휴리스틱 논리 최소화 방법. 빠르고 충분히 좋은 해를 주지만 최적임은 보장하지 않음.

단계	동작
EXPAND	각 implicant 를 prime 이 되도록 최대한 키움
IRREDUNDANT COVER	없어도 되는 prime 항을 제거해, 남은 항을 하나라도 통째로 빼면 원래 함수를 표현할 수 없는 cover를 만듦. 전체 항 수가 가장 적다는 보장은 없음.
REDUCE	cover 의 implicant 를 다시 축소(함수는 그대로). 다음 EXPAND 가 다른 prime 을 찾을 수 있게 함

- EXPAND → IRREDUNDANT COVER → REDUCE 를 반복하다가 개선이 없으면 멈춤. 상용 CAD 툴에서 가장 널리 쓰는 SW.

예 · Espresso의 REDUCE–EXPAND 반복

F(A,B,C,D) = Σm(0,1,2,5,6,7,8,9,10,14)를 사용한다. 초기 irredundant cover를 F = B′C′ + CD′ + A′C′D + A′BC로 선택할 수 있다. 이 cover는 네 개의 PI로 모든 on-set minterm을 덮으며, 어느 한 항을 제거해도 함수가 유지되지 않지만, 최소 cover는 아니다.

REDUCE 단계에서는 다른 항들이 이미 덮는 minterm을 고려하여 A′C′D를 m5에 해당하는 A′BC′D로, A′BC를 m7에 해당하는 A′BCD로 축소할 수 있다. m1은 B′C′가, m6은 CD′가 여전히 덮기 때문에 전체 함수는 유지된다.

다음 EXPAND 단계에서 m5와 m7을 나타내는 두 cube를 모두 PI인 A′BD로 확장할 수 있다. IRREDUNDANT COVER 단계에서 중복된 A′BD 하나를 제거하면 F = B′C′ + CD′ + A′BD라는 세 항의 cover가 얻어진다.

이 예제는 REDUCE로 기존 PI의 일부를 축소한 뒤 EXPAND에서 다른 PI를 선택하면 더 작은 cover를 얻을 수 있음을 보여준다. 다만 모든 반복이 반드시 항 수를 줄이거나 전역 최적해에 도달하는 것은 아니다.